



Raspodijeljene i nerelacijske baze podataka

VJEŽBA 03: Pogledi, trigeri, transakcije

Tomislav Damjanović

Transakcije

--@@TRANCOUNT funkcija vraća broj pokrenutih BEGIN TRANSACTION izjava u trenutnom sessionu

--ovu funkciju možemo koristiti za prebrojavanje lokalnog broja transakcija

```
select @@TRANCOUNT;
```

--ova funkcija vraća broj sesije

```
select @@SPID
```

--pokrenite još jedan session (New Query) i tamo provjerite broj transakcija i sesija

--pokrenite

```
select * from fn_dblog(null, null);
```

Transakcije

`select @@TRANCOUNT`

- Otvorite 2 *New Query prozora* i u svakom izvršite
- U prvom prozoru napravite upit za ažuriranje tablice `production.stocks` tako da se količina proizvoda s id-om 6 u trgovini s id-om 1 ažurira na 1
- Pripremite upit za dohvat podataka iz iste tablice s istim filterom

`select @@SPID`

```
update production.stocks
set quantity=1
where product_id=6
and store_id=1
```

Izvršite SELECT u oba prozora.
Ima li razlika?

```
select * from production.stocks
where product_id=6
and store_id=1
```

Transakcije

- Sada u prvom prozoru prije UPDATE izraze eksplicitno pokrenite transakciju i ažurirajte istu tablicu na količinu 0

```
BEGIN TRANSACTION;  
update production.stocks  
set quantity=0  
where product_id=6  
and store_id=1
```

1. U prvom prozoru dodajte i izvršite COMMIT TRANSACTION nakon ažuriranja.
Što bi se dogodilo da smo umjesto COMMIT izvršili ROLLBACK TRANSACTION?

1. Opet izvršite SELECT u oba prozora.
Ima li razlika?
2. U drugom prozoru pokrenite
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

```
select * from production.stocks  
where product_id=6  
and store_id=1
```

Zadatak

- Napravite upit koji vraća popis proizvoda na skladištu (id proizvoda, naziv , id trgovine, količinu proizvoda)

```
select
p.product_id,
p.product_name,
s.store_id,
s.quantity
from production.products p inner join
production.stocks s on
p.product_id=s.product_id
```

Kada se mijenja stanje na skladištu?
Možemo li prodati proizvod kojeg nema na skladištu?

- Pri izradi računa potrebno je „skinuti” količinu kupljenih proizvoda sa skladišta.
- Bilo bi dobro imati upit koji nam uvijek vraća takve proizvode
 - Takav upit bi mogli često koristiti kao podupit što čini upite složenijima

Pogled (View)

- Umjesto pisanja upita napraviti *view*
- *Pogled (view)* jer pohranjen upit kojim se stvara virtualna tablica
- *View* se ponaša kao tablica i može se koristiti u drugim upitima, pohranjenim procedurama, funkcijama i sl.

```
CREATE VIEW production.v_products_stock
AS
select
p.product_id,
p.product_name,
s.store_id,
s.quantity
from production.products p inner join
production.stocks s on
p.product_id=s.product_id
```

Okidač (eng. Trigger)

- Pri izradi računa potrebno je „skinuti” količinu kupljenih proizvoda sa skladišta.
- Ovakve „akcije” je najbolje automatizirati što možemo napraviti okidačem (trigger)
- **Problem:** nakon unosa nove stavke narudžbe trebamo „skinuti” (ažurirati) količinu proizvoda na skladištu

Na koju tablicu trebamo primijeniti okidač?

Pomoćni upiti

- **Napravite upite bez izvršavanja!**

1. Unos nove stavke:

- Dodajte stavku u narudžbu 1 (`order_id=1`) u kojoj dodajete jedan proizvod Surly Ice Cream Truck Frameset – 2016 (`product_id=6`) bez popusta

2. Ažurirajte skladište (`production.stocks`) tako da smanjite količinu proizvoda za unesenu za trgovinu `store_id=1`

```
sales.order_items
Columns
  order_id (PK, FK, int, not null)
  item_id (PK, int, not null)
  product_id (FK, int, not null)
  quantity (int, not null)
  list_price (decimal(10,2), not null)
  discount (decimal(4,2), not null)
```

Kako ćemo dohvatiti `item_id`?
Kako ćemo dohvatiti `list_price`?

Unos

1. Unos nove stavke (napravite upit bez izvršavanja):

- Dodajte stavku u narudžbu 1 (order_id=1) u kojoj dodajete jedan proizvod Surly Ice Cream Truck Frameset – 2016 (product_id=6) bez popusta

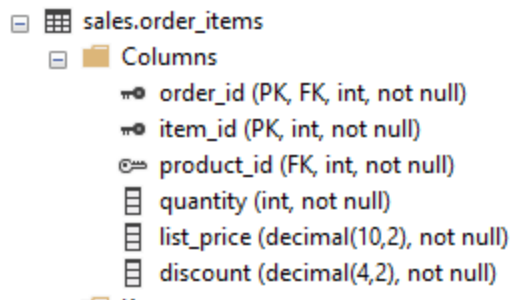


Diagram showing the structure of the sales.order_items table. It includes columns: order_id (PK, FK, int, not null), item_id (PK, int, not null), product_id (FK, int, not null), quantity (int, not null), list_price (decimal(10,2), not null), and discount (decimal(4,2), not null).

Column	Constraints
order_id	PK, FK, int, not null
item_id	PK, int, not null
product_id	FK, int, not null
quantity	int, not null
list_price	decimal(10,2), not null
discount	decimal(4,2), not null

```
insert into sales.order_items
values (
1,
(select max(item_id)+1 from sales.order_items where order_id=1),
6,
1,
(select list_price from production.products where product_id=6),
0
);
```

Ažuriranje

- Napravite upit bez izvršavanja!
- Ažurirajte skladište (production.stocks) tako da smanjite količinu proizvoda za unesenu

```
update production.stocks  
set quantity-=1  
where product_id=6  
and store_id=1;
```

Izrada okidača

[schema_name.] ->opcionalno
trigger_name ->obavezno, naziv triggera
[NOT FOR REPLICATION] – opcionalno, neće se izvršavati u slučaju procesa replikacije

- Izraz: CREATE TRIGGER

Može li BEFORE?
INSTEAD_OF

```
CREATE TRIGGER [naziv_scheme.]naziv_triggera  
ON naziv_tablice  
AFTER {[INSERT],[UPDATE],[DELETE]}  
[NOT FOR REPLICATION]  
AS  
{sql_statements}
```

Napraviti AFTER INSERT okidač u kojem će se u varijable @prod_id i @store_id učitati odgovarajuće vrijednosti. Nakon toga treba ažurirati tablicu production.stocks tako da se u određenoj trgovini umanja količina za unesenu.

Izrada okidača

Nazivima okidača
ćemo davati prefiks *trg*

```
CREATE TRIGGER sales.trg_stock_upd
ON sales.order_items
AFTER INSERT
AS
BEGIN
declare @prod_id as int;
declare @store_id as int;
set @prod_id=(SELECT i.product_id FROM inserted i)
set @store_id =
    (SELECT o.store_id FROM inserted i inner join sales.orders o on o.order_id=i.order_id)

    update production.stocks
    set quantity=quantity-(SELECT i.quantity FROM inserted i)
    where product_id=@prod_id
    and store_id=@store_id;
END
```

Deklariramo varijable u koju ćemo
dohvatiti product_id iz unešenog retka i
store_id u kojoj je narudžba provedena

Inicijaliziramo varijable na vrijednost
product_id iz unešenog retka (virtualna
tablica inserted)

Ažuriranje podataka u
tablici
production.stocks

Unos podataka

- Provjerite stanje stavki narudžbe order_id= 1 prije i nakon izvršavanja unosa.
- Provjerite stanje na skladištu za proizvod product_id prije i nakon unosa.

```
insert into sales.order_items
values (
1,
(select max(item_id)+1 from sales.order_items
where order_id=1),
6,
1,
(select list_price from production.products
where product_id=6),
0
);
```

```
select * from
production.v_products_stock
where product_id=6;
```

Zadaci

- Na stranicama kolegija preuzmite i riješite zadatke v_3 te ih predajte kao zadaću.